Chapter 3

Identifying Objects

IN THIS CHAPTER

- **Understanding classification**
- **Considering classification types**
- **Performing classification tasks**
- **Tricking image recognition software**

Deep learning solutions for image recognition have become so impressive in their human-level performance that you see them used in developing or already marketed applications, such as self-driving cars and video-surveillance appliances. The video-surveillance appliances already perform tasks, such as automatic satellite image monitoring, facial detection, and people localization and counting. Yet you can't imagine a complex application when your network labels an image with only a single prediction. Even a simple dog or cat detector may not prove useful when the photos you analyze contain multiple dogs and cats. The real world is messy and complex. You can't expect, except in limited and controlled cases, laboratory-style images that consist of single, clearly depicted objects.

The need to handle complex images paved the way for variants of Convolutional Neural Networks (CNNs). Such variants offer sophistica-

tion that's still being developed and refined, such as multiple-object detection and localization. Multiple-object detection can deal with many different objects at a time. Localization can tell you where they are in the picture, and segmentation can find their exact contours. These new capabilities require complex neural architectures and image processing more advanced than the basic CNNs discussed in previous chapters. This chapter illustrates the fundamentals of how these solutions work, names key approaches and architectures, and, finally, tests one of the best performing object detection implementations.

The chapter closes by unveiling an expected weakness in an otherwise unbelievable technology. Someone could maliciously trick CNNs to report misleading detections or ignore seen objects using appropriate image-manipulation techniques. This puzzling discovery opens a new research front that shows that deep learning performance must also consider security for private and public use.

REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0503_Object_ID.ipynb` source code file for Python and `DSPD_R_0503_Object_ID.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Distinguishing Classification Tasks

CNNs are the building blocks of deep learning–based image recognition, yet they answer only a basic classification need: Given a picture, can the CNN associate its content with a specific image class learned through previous examples? The following sections discuss the issues concerning this seemingly simple task. Humans can perform it with ease, but computers find the task difficult at best.

Understanding the problem

When you train a deep neural network to recognize dogs and cats, you can feed it a photo and obtain output that tells you whether the photo contains a dog or cat. If the last network layer is a softmax layer, the network outputs the probability of the photo containing a dog or a cat (the two classes you trained it to recognize) and the output sums to 100 percent. When the last layer is a sigmoid-activated layer, you obtain scores that you can interpret as probabilities of content belonging to each class, independently. The scores won't necessarily sum to 100 percent. In both cases, the classification may fail when the following occurs:

- **The main object isn't what you trained the network to recognize.** You may present the example neural network with a photo of a raccoon. In this case, the network will output an incorrect answer of dog or cat.
- **The main object is partially obstructed.** Your cat is playing hide and seek in the photo you show the network, and the network can't spot the cat because a piece of furniture partially hides it.
- **The photo contains many different objects to detect.** The image may include animals other than cats and dogs. In this case, the output from the network will suggest a single class rather than include all the objects.

Figure 3-1 shows image 47780 (`http://cocodataset.org/#explore?id=47780`) taken from the MS Coco dataset (released as part of the open source Creative Commons Attribution 4.0 License). The series of three outputs shows how a CNN detects, localizes, and segments the objects appearing in the image (a kitten and a dog standing on a field of grass). A plain CNN can't reproduce the examples in Figure 3-1 because its architecture will treat the entire image as being of a certain class. To overcome this limitation, researchers extend the basic CNN's capabilities to make them capable of the following:

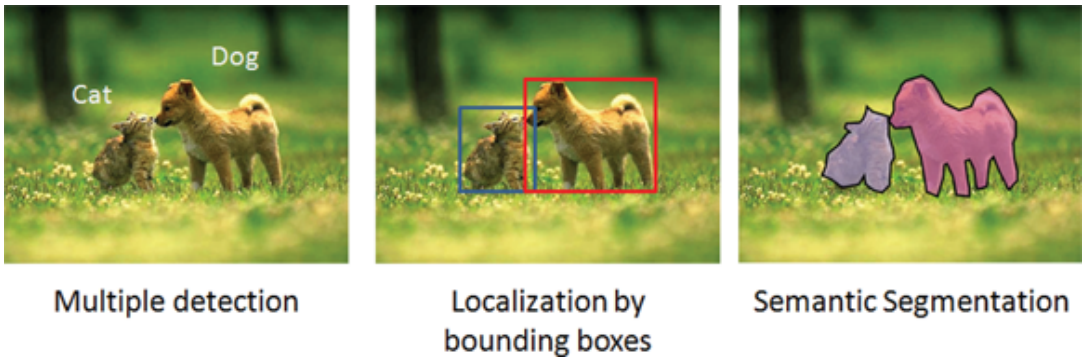

FIGURE 3-1: Detection, localization, and segmentation example from the Coco dataset.

- **Detection:** Determining when an object is present in an image. Detection is different from classification because it involves just a portion of the image, implying that the network can detect multiple objects of the same and of different types. The capability to spot objects in partial images is called *instance spotting*.
- **Localization:** Defining exactly where a detected object appears in an image. You can have different types of localizations. Depending on granularity, they distinguish the part of the image that contains the detected object.
- **Segmentation:** Classification of objects at the pixel level. Segmentation takes localization to the extreme. This kind of neural model assigns each pixel of the image to a class or even an entity. For instance, the network marks all the pixels in a picture relative to dogs and distinguishes each one using a different label (called *instance segmentation*).

Performing localization

Localization is perhaps the easiest extension that you can get from a regular CNN. It requires that you train a regressor model alongside your deep learning classification model. A *regressor* is a model that guesses numbers. Defining object location in an image is possible using corner pixel coordinates, which means that you can train a neural network to output key measures that make it easy to determine where the classified object appears in the picture using a bounding box. Usually a bounding box uses the x and y coordinates of the lower-left corner, together with the width and the height of the area that encloses the object.

Classifying multiple objects

The classification of multiple objects begins by classifying those objects individually. A CNN performs these tasks on each object:

- **Detect:** Predict an object's class.
- **Localize:** Provide the object's coordinates.

You still use a CNN to classify multiple objects in an image, but this means working with each object present in the picture individually using one of two old image-processing solutions:

- **Sliding window:** Analyzes only a portion (called a *region of interest*) of the image at a time. When the region of interest is small enough, it likely contains only a single object. The small region of interest allows the CNN to correctly classify the object. This technique is called *sliding window* because the software uses an image window to limit visibility to a particular area (the way a window in a home does) and slowly moves this window around the image. The technique is effective but could detect the same image multiple times, or you may find that some objects go undetected based on the window size that you decide to use to analyze the images.
- **Image pyramids:** Solves the problem of using a window of fixed size because it generates increasingly smaller resolutions of the image. Therefore, you can apply a small sliding window. In this way, you transform the objects in the image, and one of the reductions may fit exactly into the sliding window used.

These techniques are computationally intensive. To apply them, you have to resize the image multiple times and then split it into chunks. You then process each chunk using your classification CNN. The number of operations for these activities is so large that rendering the output in real time is impossible.

The sliding window and image pyramid have inspired deep learning researchers to discover a couple of conceptually similar approaches that are less computationally intensive. The first approach is *one-stage detection.* It works by dividing the images into grids, and the neural

network makes a prediction for every grid cell, predicting the class of the object inside. The prediction is quite rough, depending on the grid resolution (the higher the resolution, the more complex and slower the deep learning network). One-stage detection is very fast, having almost the same speed as a simple CNN for classification. The results have to be processed to gather the cells representing the same object together, and that may lead to further inaccuracies. Neural architectures based on this approach are Single-Shot Detector (SSD), You Only Look Once (YOLO), and RetinaNet. One-stage detectors are very fast, but not so precise.

The second approach is *two-stage detection.* This approach uses a second neural network to refine the predictions of the first one. The first stage is the proposal network, which outputs its predictions on a grid. The second stage fine-tunes these proposals and outputs a final detection and localization of the objects. Region Convolutional Neural Network (R-CNN), Fast R-CNN, and Faster R-CNN are all two-stage detection models that are much slower than their one-stage equivalents, but more precise in their predictions. You can read a more in-depth version of the technology behind R-CNN, Fast R-CNN, and Faster R-CNN at `https://towardsdatascience.com/r-cnn-fast-r-cnn-faster-r-cnn-yolo-object-detection-algorithms-36d53571365e`.

Annotating multiple objects in images

To train deep learning models to detect multiple objects, you need to provide more information than in simple classification. For each object, you provide both a classification and coordinates within the image using the annotation process, which contrasts with the labeling used in simple image classification.

Labeling images in a dataset is a daunting task even in simple classification. Given a picture, the network must provide a correct classification for the training and test phases. In labeling, the network decides on the right label for each picture, and not every part of the network

will perceive the depicted image in the same way. The people who created the ImageNet dataset used the classification provided by multiple users from the Amazon Mechanical Turk crowdsourcing platform. (ImageNet used the Amazon service so much that in 2012, it turned out to be Amazon's most important academic customer.)

In a similar way, you rely on the work of multiple people when annotating an image using bounding boxes. Annotation requires that you not only label each object in a picture but also determine the best box with which to enclose each object. These two tasks make the annotation even more complex than labeling and more prone to producing erroneous results. Performing annotation correctly requires the work of more people who can provide a consensus on the accuracy of the annotation.

TIP Some open source software can help in annotation for image detection (as well as for image segmentation, discussed in the following section). These tools are particularly effective:

- **LabelImg (** `https://github.com/tzutalin/labelImg` **):** Created by TzuTa Lin using Python, it relies on the Qt graphical interface (`https://doc.qt.io/qt-5.9/qtgui-index.html`). You can find a tutorial for this tool at `https://www.youtube.com/watch?v=p0nR2YsCY_U`).
- **LabelMe (** `https://github.com/wkentaro/labelme` **):** A powerful tool for image segmentation that provides an online service. The advantage of this tool is that you can use more than just squares to build a box to enclose each object, which makes it more time consuming to use but also more accurate.
- **FastAnnotationTool (** `https://github.com/christopher5106/FastAnnotationTool` **):** Based on the OpenCV computer vision library (`https://developer.nvidia.com/opencv`). This package isn't maintained as well as the others in the list but is still viable.

Segmenting images

Semantic segmentation predicts a class for each pixel in the image, which is a different perspective from either labeling or annotation. Some people also call this task *dense prediction* because it makes a prediction for every pixel in an image. The task doesn't specifically distinguish different objects in the prediction. For instance, a semantic segmentation can show all the pixels that are of the class `cat`, but it won't provide any information about what the cat (or cats) is doing in the picture. You can easily get all the objects in a segmented image by *postprocessing*, because after performing the prediction, you can get the object pixel areas and distinguish between different instances of them, if multiple separated areas exist under the same class prediction.

Different deep learning architectures can achieve image segmentation. Fully Convolutional Networks (FCNs) and U-Nets (`https://arxiv.org/abs/1505.04597`) are among the most effective. FCNs are built for the first part (called the *encoder*), which is the same as CNNs. After the initial series of convolutional layers, FCNs end with another series of CNNs that operate in a reverse fashion as the encoder (making them a *decoder*). The decoder is constructed to re-create the original input image size and output as pixels the classification of each pixel in the image. In such a fashion, the FCN achieves the semantic segmentation of the image. FCNs are too computationally intensive for most real-time applications. In addition, they require large training sets to learn their tasks well; otherwise, their segmentation results are often coarse.

REMEMBER Finding the encoder part of the FCN pretrained on ImageNet, which accelerates training and improves learning performance, is common.

U-Nets are an evolution of FCN devised by Olaf Ronneberger, Philipp Fischer, and Thomas Brox in 2015 for medical purposes (see

`https://lmb.informatik.uni-freiburg.de/people/ronneber/U-Net/`). U-Nets present advantages compared to FCNs. The encoding (also called *contraction*) and the decoding parts (also referred to as *expansion*) are perfectly symmetric. In addition, U-Nets use shortcut connections between the encoder and the decoder layers. These shortcuts allow the details of objects to pass easily from the encoding to the decoding parts of the U-Net, and the resulting segmentation is precise and fine-grained.

TIP Building a segmentation model from scratch can be a daunting task, but you don't need to do that. You can use some pretrained U-Net architectures and immediately start using this kind of neural network by leveraging the segmentation *model zoo* (a term used to describe the collection of pretrained models offered by many frameworks; see `https://modelzoo.co/` for details) offered by segmentation models, a package offered by Pavel Yakubovskiy. You can find installation instructions, the source code, and plenty of usage examples at `https://github.com/qubvel/segmentation_models` . The commands from the package seamlessly integrate with Keras.

Perceiving Objects in Their Surroundings

As automation takes a larger role in all aspects of human existence, the need for automation to perceive objects becomes greater. Robots need to see obstacles, self-driving cars need to see pedestrians, and even medical systems need to see patients. The computer that controls the automation can't understand what an object is or how it functions; instead, it uses camera input in the form of digital images to identify objects and then interact with those objects in predefined ways. The following sections discuss how automation uses this form of perception (which differs from human perception) through 2-D camera imagery to perform tasks.

Considering vision needs in self-driving cars

Integrating vision capabilities into the sensing system of a self-driving car could enhance how safely it drives. A segmentation algorithm could help the car distinguish lanes from sidewalks, as well as from other obstacles the car should notice. The car could even feature a complete end-to-end system, such as NVidia's, that controls steering, acceleration, and braking in a reactive manner based on its visual inputs. (You can learn more about the NVidia self-driving car efforts at `https://www.nvidia.com/en-us/self-driving-cars/` .) A visual system could spot certain objects on the road relevant to driving, such as traffic signs and traffic lights. It could visually track the trajectories of other cars. In all cases, a deep learning network could provide the solution.

REMEMBER No concept of emotion exists for a computer, so a computer performs an analysis and then follows rules to perform tasks such as steering. A computer can't drive confidently because confidence is an expression of an emotional state brought on by success. A successful computer only performs its tasks with greater precision. The lack of emotion also means that computers don't experience an adrenaline rush prior to an accident, making the capability to act faster than normal a reality. A computer can make decisions at the one speed at which analysis allows it to make decisions. In short, no matter how well you program a computer to mimic human processes, the computer will still perform differently from a human and react to environmental issues in different ways.

The "Distinguishing Classification Tasks" section, at the start of this chapter, discusses how object detection improves upon single-object classification offered by CNNs. This section also clarifies the architectures and current models of the two main approaches: one-stage detection (or one-shot detection) and two-stage detection (also known as

region proposal). This section tells how a one-stage detection system works and provides help for an autonomous vehicle. You need to consider these issues in light of how computers differ from humans when driving a car, but you can apply the principles to any form of automation where a computer takes over a task from a human.

Programming such a detection system from scratch would be a daunting task, one requiring an entire book of its own. Fortunately, you can employ open source projects on GitHub such as Keras-RetinaNet (`https://github.com/fizyr/keras-retinanet`). The next section of the chapter provides additional details on how using RetinaNet works.

TIP Isaac Newton stated, "If I have seen further, it is by standing on the shoulders of Giants." Likewise, you can achieve more in deep learning when you make use of existing neural architectures and pretrained networks. For instance, you can find many models on GitHub (`www.github.com`) such as the TensorFlow model zoo (`https://github.com/tensorflow/models`).

Discovering how RetinaNet works

The RetinaNet is a sophisticated and interesting object-detection model that strives to be as fast as other one-stage detection models while also achieving the accuracy of bounding box predictions of two-stage detection systems like Faster R-CNN (the top-performing model). Thanks to its architecture, RetinaNet achieves its goals, using techniques similar to the U-Net architecture discussed for semantic segmentation. RetinaNet is part of a group of models called Feature Pyramid Networks (FPN).

RetinaNet owes its performance to its authors, Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár, who noted that one-stage detection models don't always detect objects precisely be-

cause they are affected by the overwhelming presence of distracting elements in the images used for training. Their paper, "Focal Loss for Dense Object Detection" (`https://arxiv.org/pdf/1708.02002.pdf`), provides details of the techniques RetinaNet uses. The problem is that the images present few objects of interest to detect. In fact, one-stage detection networks are trained to guess the class of each cell in an image divided by a fixed grid, where the majority of cells are empty of objects of interest.

REMEMBER In semantic segmentation, the targets of the classification are single pixels. In one-stage detection, the targets are sets of contiguous pixels, performing a task similar to semantic segmentation but at a different granularity level.

Here's what happens when you have such a predominance of null examples in images and are using a training approach that examines all available cells as examples. The network will be more likely to predict that nothing is in a processed image cell than to provide a correctly predicted class. Neural networks always take the most efficient route to learn, and in this case, predicting the background is easier than anything else. In this situation, which goes under the name of *unbalanced learning*, many objects are undetected by the neural network using a single-shot object detection approach.

In machine learning, when you want to predict two numerically different classes (one is the majority class and the other is the minority class), you have an unbalanced classification problem. Most algorithms don't perform properly when the classes are unbalanced because they tend to prefer the majority class. A few solutions are available for this problem:

- **Sampling:** Selecting some examples and discarding others.

- **Downsample:** Reducing the effect of the majority class by choosing to use only a part of it, which balances the majority and minority predictions. In many cases, this is the easiest approach.
- **Upsample:** Increasing the effect of the minority class by replicating its examples many times until the minority class has the same number of examples as the majority class.

The creators of RetinaNet take a different route, as they note in their paper "Focal Loss for Dense Object Detection" mentioned earlier in this section. They discount the majority class examples that are easier to classify and concentrate on the cells that are difficult to classify. The result is that the network cost function focuses more on adapting its weights to recognize background objects. This is the *focal loss* solution and represents a smart way to make one-stage detection perform more correctly, yet speedily, which is a real-time application requirement, such as for obstacle or object detection in self-driving cars, or processing large quantities of images in video surveillance.

Using the Keras-RetinaNet code

Released under the open source Apache License 2.0, Keras-RetinaNet is a project sponsored by the Dutch robotic company Fitz and made possible by many contributors (the top contributors are Hans Gaiser and Maarten de Vries). It's an implementation of the RetinaNet neural network written in Python using Keras (`https://github.com/fizyr/keras-retinanet/`). You find Keras-RetinaNet used successfully by many projects — the most notable and impressive of which is the winning model for the NATO Innovation Challenge, a competition whose task was to detect cars in aerial images. (You can read the narrative from the winning team in this blog post: `https://medium.com/data-from-the-trenches/object-detection-with-deep-learning-on-aerial-imagery-2465078db8a9`.)

Object detection network code is too complex to explain in a few pages, plus you can use an existing network to set up deep learning solutions, so this section demonstrates how to download and use Keras-

RetinaNet on your computer. Before you try this process, ensure that you have configured your computer as described in Book 1, Chapter 5, and consider the trade-offs involved in using various execution options described in the "Considering the cost of realistic output" sidebar in Book 5, Chapter 2.

Obtaining Keras-RetinaNet

As a first step, you upload the necessary packages and start downloading the zipped version of the GitHub repository. This example uses the 0.5.0 version of Keras-RetinaNet, which was the most recent version available at the time of writing.

```
import os
import zipfile
import urllib.request
import warnings
warnings.filterwarnings("ignore")
url = "https://github.com/fizyr/\
keras-retinanet/archive/0.5.0.zip"
urllib.request.urlretrieve(url, './'+url.split('/')[-1])
```

After downloading the zipped code, the example code automatically extracts it using these commands:

```
zip_ref = zipfile.ZipFile('./0.5.0.zip', 'r')
for name in zip_ref.namelist():
  zip_ref.extract(name, './')
```

```
zip_ref.close()
```

The execution creates a new directory called `keras-retinanet-0.5.0`, which contains the code for setting up the neural network. The code then executes the compilation and installation of the package using the `pip` command:

```
os.chdir('./keras-retinanet-0.5.0')
```

```
!python setup.py build_ext --inplace
```

```
!pip install .
```

As the setup process proceeds, you see a long series of messages as output. The task can also require a little time to complete. Both the messages and the installation time are normal. The best thing you can do is get a cup of coffee and wait.

Obtaining pretrained weights

All the commands in the previous section retrieved the code that builds the network architecture. The example now needs the pretrained weights and relies on weights trained on the MS Coco dataset using the ResNet50 CNN, the neural network that Microsoft used to win the 2015 ImageNet competition.

```
os.chdir('../')
```

```
url = "https://github.com/fizyr/keras-retinanet/\
```

```
releases/download/0.5.0/resnet50_coco_best_v2.1.0.h5"
```

```
urllib.request.urlretrieve(url, './'+url.split('/')[-1])
```

Downloading all the weights takes a while, so now would be a good time to catch up on your reading (assuming you don't need more

coffee).

Initializing the RetinaNet model

After you have completed the downloads, the example is ready to import all the necessary commands and to initialize the RetinaNet model using the pretrained weights retrieved from the Internet. This step also sets a dictionary to convert the numeric network results into understandable classes. The selection of classes is useful for the detector on a self-driving car or any other solution that has to understand images taken from a road or an intersection.

```
import os

import numpy as np

from collections import defaultdict

import keras

from keras_retinanet import models

from keras_retinanet.utils.image import (read_image_bgr,

          preprocess_image, resize_image)

from keras_retinanet.utils.visualization import (draw_box,

          draw_caption)

from keras_retinanet.utils.colors import label_color

import matplotlib.pyplot as plt

%matplotlib inline

```

```
model_path = os.path.join('.',
          'resnet50_coco_best_v2.1.0.h5')

model = models.load_model(model_path,
          backbone_name='resnet50')

labels_to_names = defaultdict(lambda: 'object',
         {0: 'person', 1: 'bicycle', 2: 'car',
          3: 'motorcycle', 4: 'airplane', 5: 'bus',
          6: 'train', 7: 'truck', 8: 'boat',
          9: 'traffic light', 10: 'fire hydrant',
          11: 'stop sign', 12: 'parking meter',
          25: 'umbrella'})
```

Downloading the test image

To make the example useful, you need a sample image to test the RetinaNet model. The example relies on a free image from Wikimedia representing an intersection with people expecting to cross the road, some stopped vehicles, traffic lights, and traffic signs.

```
url = "https://upload.wikimedia.org/wikipedia/commons/\
thumb/f/f8/Woman_with_blue_parasol_at_intersection.png/\
```

```
640px-Woman_with_blue_parasol_at_intersection.png"
```

```
urllib.request.urlretrieve(url, './'+url.split('/')[-1])
```

Testing the neural network

Now that the download process and initialization are complete, it's time to test the neural network. In the code snippet that follows this explanation, the code reads the image from disk and then switches the blue with red image channels (because the image is uploaded in BGR format, but RetinaNet works with RGB images). Finally, the code pre-processes and resizes the image. All these steps complete using the provided functions and require no special settings.

The model will output the detected bounding boxes, the level of confidence (a probability score that the network truly detected something), and a code label that will convert into text using the previously defined dictionary of labels. The loop filters the boxes printed on the image by the example. The code uses a confidence threshold of 0.5, implying that the example will keep any detection whose confidence is at least at 50 percent. Using a lower confidence threshold results in more detections, especially of those objects that appear small in the image, but also increases wrong detections (for instance, some shadows may start being detected as objects).

TIP Depending on your objectives using RetinaNet, you may decide that using a lower confidence threshold is fine. You'll notice that as you lower the confidence, the proportion of the resulting exact guesses (those with nearly 100 percent confidence) will diminish. Such a proportion is called the *precision* of the detection, and by deciding what precision you can tolerate, you can set the best confidence for your purposes.

```
image = read_image_bgr(
    '640px-Woman_with_blue_parasol_at_intersection.png')
draw = image.copy()
draw[:,:,0], draw[:,:,2] = image[:,:,2], image[:,:,0]

image = preprocess_image(image)
image, scale = resize_image(image)

boxes, scores, labels = model.predict_on_batch(
    np.expand_dims(image, axis=0))
boxes /= scale

for box, score, label in zip(
    boxes[0], scores[0], labels[0]):

    if score > 0.5:
      color = label_color(label)
      b = box.astype(int)
      draw_box(draw, b, color=color)
```

```
        caption = "{} {:.3f}".format(
            labels_to_names[label], score)
        draw_caption(draw, b, caption.upper())
plt.figure(figsize=(12, 6))
plt.axis('off')
plt.imshow(draw)
plt.show()
```

The first time you run the code may take a while, but after some computations, you should obtain the output reproduced in Figure 3-2.

FIGURE 3-2: Object detection resulting from Keras-RetinaNet.

The network can successfully detect various objects, some extremely small (such as a person in the background), some partially shown (such as the nose of a car on the right of the image). Each detected object is delimited by its bounding box, which creates a large range of possible applications.

For instance, you could use the network to detect that an umbrella — or some object — is being used by a person. When processing the results, you can relate the fact that two bounding boxes are overlapping, with one being an umbrella and the other one being a person, and that the first box is positioned on top of the second in order to infer that a person is holding an umbrella. This is called *visual relationship detection*. In the same way, by the overall setting of detected objects and their relative positions, you can train a second deep learning network to infer an overall description of the scene.

Overcoming Adversarial Attacks on Deep Learning Applications

As deep learning finds many applications in self-driving cars, such as detecting and interpreting traffic signs and lights; detecting the road and its lanes; detecting crossing pedestrians and other vehicles; controlling the car by steering and braking in an end-to-end approach to automatic driving; and so on, questions may arise about the safety of a self-driving car. Driving isn't the only common activity that's undergoing a revolution because of deep learning applications. Recently introduced applications that are accessible by the public include facial recognition for security access. (You can read about this use in ATMs in China at `https://www.telegraph.co.uk/news/worldnews/asia/china/11643314/China-unveils-worlds-first-facial-recognition-ATM.html`.) Another example of a deep learning application is in speech recognition used for Voice Controllable Systems (VCSs), as provided by a plethora of companies such as Apple, Amazon, Microsoft, and Google in a wide variety of applications that include Siri, Alexa, and Google Home.

Some of these deep learning applications may cause economic damage or even be life threatening when they fail to provide the correct answer. Therefore, you may be surprised to discover that hackers can intentionally trick deep neural networks and guide them into failing

predictions by using particular techniques called adversarial examples.

An *adversarial example* is a handcrafted piece of data that is processed by a neural network as training or test inputs. A hacker modifies the data to force the algorithm to fail in its task. Each adversarial example bears modifications that are indeed slight, subtle, and deliberately made imperceptible to humans. The modifications, although ineffective on humans, are still quite effective in reducing the effectiveness and usefulness of a neural network. Often, such malicious examples aim at leading a neural network to fail in a predictable way to create some illegal advantage for the hacker. Here are just a few malicious uses of adversarial examples (the list is far from exhaustive):

- Misleading a self-driving car into an accident
- Obtaining money from an insurance fraud by having fake claim photos trusted as true ones by automatic systems
- Tricking a facial recognition system to recognize the wrong face and grant access to money in a bank account or personal data on a mobile device

Tricking pixels

First made known by the paper "Intriguing Properties of Neural Networks" (go to `https://arxiv.org/pdf/1312.6199.pdf`), adversarial examples have attracted much attention in recent years, and successful (and shocking) discoveries in the field have led many researchers to devise faster and more effective ways of creating such examples than the original paper pointed out.

DISCOVERING THAT A MUFFIN IS NOT A CHIHUAHUA

Sometimes deep learning image classification fails to provide the right answer because the target image is inherently ambiguous or rendered to puzzle observers. For instance, some images are so misleading that they can even mystify a human examiner for a while, such as the Internet memes Chihuahua vs Muffin (see

`https://imgur.com/QWQiBYU`) or Labradoodle vs Fried Chicken (see `https://imgur.com/5EnWOJU`). A neural network can misunderstand confusing images if its architecture isn't adequate to the task and its training hasn't been exhaustive in terms of seen examples. The AI technology columnist Mariya Yao has compared different computer vision APIs at `https://medium.freecodecamp.org/chihuahua-or-muffin-my-search-for-the-best-computer-vision-api-cbda4d6b425d`) and found that even full-fledged vision products can be tricked by ambiguous pictures.

Recently, other studies have challenged deep neural networks by proposing unexpected perspectives of known objects. In the paper called "Strike (with) a Pose: Neural Networks Are Easily Fooled by Strange Poses of Familiar Objects" at `https://arxiv.org/pdf/1811.11553.pdf`, researchers found that simple ambiguity can trick state-of-the-art image classifiers and object detectors trained on large-scale image datasets.

Often, objects are learned by neural networks from pictures taken in *canonical poses* (which means in common and usual situations). When faced with an object in an unusual pose or outside its usual environment, some neural networks can't categorize the resulting object. For instance, you expect a school bus to be running on the road, but if you rotate and twist it in the air and then land it in the middle of the road, a neural network can easily see it as a garbage truck, a punching bag, or even a snowplow. You may argue that the misclassification occurs because of learning bias (teaching a neural network using only images in canonical poses). Yet that implies that at present, you shouldn't rely such technology under all circumstances, especially, as the authors of the paper point out, in self-driving car applications because objects may suddenly appear on the road in new poses or circumstances.

REMEMBER Adversarial examples are still confined to deep learning research laboratories. For this reason, you find many scientific papers quoted in these paragraphs when referring to various kinds of examples. However, you should never discount adversarial examples as being some kind of academic diversion because their potential for damage is high.

At the foundations of all these approaches the idea that mixing some numeric information, called a perturbation, with the image can lead a neural network to behave differently from expectations, although in a controlled way. When you create an adversarial example, you add some specially devised noise (which appear to be random numbers when you view them) to an existing image, and that's enough to trick most CNNs (because often the same trick works with different architectures when trained by the same data). Generally, you can discover such perturbations by having access to the model (its architecture and weights). You then exploit its backpropagation algorithm to systematically discover the best set of numeric information to add to an image so that you can mutate one predicted class into another one.

TIP You can create the perturbation effect by changing a single pixel in an image. Researchers have obtained perfectly working adversarial examples using this approach, as discovered by researchers Jiawei Su, Danilo Vasconcellos Vargas, and Kouichi Sakurai from Kyushu University and described in their paper "One Pixel Attack for Fooling Deep Neural Networks" (`https://arxiv.org/pdf/1710.08864.pdf`).

Hacking with stickers and other artifacts

Most adversarial examples are laboratory experiments on vision robustness, and those examples can demonstrate all their capabilities because they are produced by directly modifying data inputs and tested images during the training phase. However, many applications based on deep learning operate in the real world, and the use of laboratory techniques doesn't prevent malicious attacks. Such attacks don't need access to the underlying neural model to be effective. Some examples may take the form of a sticker or an inaudible sound that the neural network doesn't know how to handle.

A paper called "Adversarial Examples in the Physical World" by Alexey Kurakin, Ian J. Goodfellow, and Samy Bengio (found at `https://arxiv.org/pdf/1607.02533.pdf`) demonstrates that various attacks are also possible in a nonlaboratory setting. All you need is to print the adversarial examples and show them to the camera feeding the neural network (for instance, by using the camera in a mobile phone). This approach demonstrates that the efficacy of an adversarial example is not strictly due to the numerical input fed into a neural network. It's the ensemble of shapes, colors, and contrast present in the image that achieves the trick, and you don't need any direct access to the neural model to find out what ensemble works best. You can see how a network could mistake the image of a washing machine for a safe or a loudspeaker directly from this video made by the authors, who tricked the TensorFlow camera demo, an application for mobile devices that performs on-the-fly image classification: `https://www.youtube.com/watch?v=zQ_uMenoBCk`.

Other researchers from Carnegie Mellon University have found a way to trick face detection into believing a person is a celebrity by fabricating eyeglass frames that can affect how a deep neural network recognizes instances. As automated security systems become widespread, the ability to trick the system by using simple add-ons like eyeglasses could turn into a serious security threat. A paper called "Accessorize to a Crime: Real and Stealthy Attacks on State-of-the-Art Face Recognition" (`https://www.cs.cmu.edu/~sbhagava/papers/face-`

`rec-ccs16.pdf`) describes how accessories could allow both dodging personal recognition and impersonation.

Finally, another disturbing real-world use of an adversarial example appears in the paper "Robust Physical-World Attacks on Deep Learning Visual Classification" (`https://arxiv.org/pdf/1707.08945.pdf`). Plain black-and-white stickers placed on a stop sign can affect how a self-driving car understands the signal, causing it to see the stop sign as another road indication. When you use more colorful (but also more noticeable) stickers, such as the ones described in the paper "Adversarial Patch" (`https://arxiv.org/pdf/1712.09665.pdf`), you can guide the predictions of a neural network in a particular direction by having it ignore anything but the sticker and its misleading information. As explained in the paper, a neural network could predict a banana to be anything else just by placing a proper deceitful sticker nearby.

At this point, you may wonder whether any defense against adversarial examples is possible, or if sooner or later they will destroy the public confidence in deep learning applications, especially in the self-driving car field. By intensely studying how to mislead a neural network, researchers are also finding how to protect it against any misuse. First, neural networks can approximate any function. If the neural networks are complex enough, they can also determine by themselves how to rule out adversarial examples when taught by other examples. Second, novel techniques such as constraining the values in a neural network or reducing the neural network size after training it (a technique called *distillation*, used previously to make a network viable on devices with little memory) have been successfully tested against many different kinds of adversarial attacks.

Table of contents collapsed